Habit Tracker

Object Oriented and Functional Programming with Python

Author: Benjamin Herrmann IU14136471

GitHub Repository:

https://github.com/BenjaminH1992/habit-tracker

1. Project Overview

The goal of this project was to design and implement a modular, backend-focused habit tracking application using Python. The application allows users to create daily and weekly habits, record check-offs, calculate streaks and analyse progress over time to earn rewards. Persistence is ensured through a SQLite database, and analytical functionality is implemented using functional programming principles.

The system was developed as a command-line interface (CLI) application with a clear separation of concerns between user interaction, persistence, and analytics logic.

2. Technical Approach & Architecture

The application follows a modular architecture consisting of:

- CLI module (user interaction and workflow control)
- Repository layer (database access and persistence logic)
- Analytics module (pure functional streak and reward calculations)
- Habit domain class (data model representation)

The project initially started with CSV-based persistence. During development, this approach was replaced with SQLite to support relational queries, scalability, and improved data integrity. The transition significantly improved maintainability and analytical flexibility.

SQLite was chosen because it is lightweight, requires no external dependencies, and integrates seamlessly with Python's built-in sqlite3 module.

Functional programming techniques were used in the analytics module to ensure pure, testable functions such as:

- current_streak()
- longest_streak_for_habit()
- convert_sparkles()

These functions operate only on stored data without modifying the database state.

3. Development Evolution

The development process evolved iteratively.

Initially, the application was focused on basic habit creation and tacking. Early usability improvements included:

- Asking the user for a name on first launch and storing it
- Displaying a personalized greeting on each startup

- Implementing a seamless menu loop for intuitive navigation
- Adding pause functionality after displaying analytical results to improve readability

A reward system was introduced to increase motivation. Streak-based rewards convert accumulated “sparkles” into stars and superstars, reinforcing user engagement.

As the project progressed, the architecture was refactored to ensure clean modular separation and improved maintainability.

Automated unit testing using pytest was implemented early to validate both repository behaviour and analytical calculations. Edge cases such as empty datasets and gaps in streaks were explicitly tested. An in-memory SQLite database was used during testing to avoid modifying production data.

Dummy data generation scripts were created to simulate four weeks of check-off activity, supporting development and demonstration scenarios.

4. Challenges & Lessons Learned

Several technical challenges emerged during development:

- Handling relative imports correctly when restructuring into a package
- Designing streak calculations for both daily and weekly periodicities
- Ensuring that analytics functions remained pure and independent form database logic
- Designing test that cover realistic edge cases

One major improvement over the initial plan was replacing CSV storage with SQLite. While CSV files were simpler, they lacked relational capabilities and were less robust for long-term scalability.

The importance of modular architecture became evident when expanding features such as rewards and analytics without modifying core logic.

5. Value & Reflection

The final application demonstrates:

- Clean modular design
- Persistent data storage
- Functional analytics
- Automated testing with edge case coverage
- Extensibility for future GUI or web-based interfaces

The separation of CLI, repository and analytics layers allows future integration of alternative user interfaces without modifying the core logic.

The project successfully combines object-oriented and functional programming concepts in a practical, maintainable application.

Overall, the most valuable outcome of this project was learning how architectural decisions directly influence scalability, testability, and extensibility.